Bayesian zero-inflated generalized Poisson regression model: estimation and case influence diagnostics

Yuting Liu, Qiankang Wang, Xi Zhang

03/04/2022

Abstract

In this paper, we mainly study the zero-inflated generalized Poisson (ZIGP) regression model and examine its analysis from a Bayesian point of view by using Markov Chain Monte Carlo methods. This model offers a way of modelling the excess zeros in addition to avoiding the over dispersed or under dispersed relative to Poisson. Meanwhile, the model selection criteria and Bayesian case influence diagnostics based on the Kullback-Leibler divergence are studied. Furthermore, to illustrate the performance of the developed methodology we apply the real word examples, we select the data set "Couple" to analyze. The author of the original literature gave results of the analysis but the code and the tools about the analysis are unpublished and private. In our study,we use R program to analysis the same data set and approach the similar results in the end.

Introduction

The Poisson regression model is the most common model used for analysis of the count data. However, a common practical problem with observed count data is the presence of excess zeros relative to the assumed count distribution. Count data are common in various areas, such as public health, agriculture, insurance, and psychology. When the sample variance is either larger or smaller than the sample mean, this means that the generalized Poisson (GP) can tackle over- or under-dispersion relative to Poisson. For count data with an excess of zeros, although a zero- inflated model can work perfectly well with an excess of zeros they cannot simultaneously deal with both zero-deflated and over- or under- dispersed data. In this paper, we study the Bayesian analysis for the zero-inflated generalized Poisson (ZIGP) regression model to address this problem. The main objective of this study is to develop diagnostic measures from a Bayesian perspective based on the Kullback-Leibler (K-L) divergence for ZIGP regression models. Firstly, we develop the ZIGP regression model based on the probability mass function of generalized Poisson distribution. Secondly, we discuss the Bayesian model selection criteria to determine the optimal model. And study the Bayesian case influence diagnostics based on the K-L divergence. Thirdly, we select a real psychological example with the data set "Couple" which is a part of the Interdisciplinary Project for the Optimization of Separation trajectories conducted in Flanders. We use the R program to investigate the impact of "education level" and "level of anxious attachment" on the number of unwanted pursuit behavior (UPB) perpetrators in the context of couple separation trajectories. In the end, we compare the results with the article and comments on the conclusion.

Methods and Analysis

Methods

1. Generalized Poisson(GP) Distribution

Taking Poisson distribution as a special case, the GP distribution can contain overdispersion or underdispersion related to Poisson, based on the range of positive or negative values of the second parameter. A random variable Y is defined to have a GP distribution if it has the following probability mass function:

$$f(y;\mu,\alpha) = \frac{1}{y!}(\frac{\mu}{1+\alpha\mu})^y(1+\alpha y)^{y-1}exp(-\frac{\mu(1+\alpha y)}{1+\alpha\mu}), y = 0, 1, 2, \cdots$$

Here, $\mu = E(Y)$ and α is a dispersion parameter. If $\alpha = 0$, this probability function will be a Poisson distribution function. When $\alpha > 0$, this model contains overdispersion, and, oppositely, when $\alpha < 0$, the model contains underdispersion and the value of α should satisfy $1 + \alpha\mu > 0$ and $1 + \alpha y > 0$ so that the probability fucntion will have non-negative values.

2. Zero-Inflated Generalized Possion(ZIGP) Regression Model

In ZIGP regression model, the independent response variable Y_i, where $i = 1, 2, \cdots, n$ has the following probability mass function:

$$P(Y_i = y_i) = \begin{cases} \phi_i + (1-\phi_i)f(0;\mu_i,\alpha) & , y_i = 0 \\ (1-\phi_i)f(y_i;\mu_i,\alpha) & , y_i = 1, 2, \cdots \end{cases}$$

Here, ϕ_i is the zero-inflation (deflation) parameter and it can be negative. In many publications, the choices of $\log\mu_i$ and the logistic transformation of ϕ_i are considered as linear combinations of covariates X_i and W_i. In this project, we use:

$$\log(\mu_i) = X_i^T\beta, \text{ and } \text{logit}(\phi_i) = \log(\frac{\phi_i}{1-\phi_i}) = W_i^T\gamma$$

, where $\beta = (\beta_1, \cdots, \beta_p)^T$ and $\gamma = (\gamma_1, \cdots, \gamma_q)^T$ are unknown regression coefficients.

With $y = (y_1, \cdots, y_n)^T$, $x = (X_1, \cdots, X_n)^T$, $w = (W_1, \cdots, W_n)^T$ and $\theta = (\alpha, \beta^T, \gamma^T)^T$, we can get the complete likelihood function for θ:

$$\begin{aligned} L(\theta|y,x,w) &= \Pi_{i=1}^n p(y_i|\theta, X_i, W_i) \\ &= \Pi_{i=1}^n(\phi_i + (1-\phi_i)f(0;\mu_i,\alpha))^{I_{y_i=0}}((1-\phi_i)f(y_i;\mu_i,\alpha))^{I_{y_i>0}} \end{aligned}$$

3. Bayesian Inference

3.1 *Prior Distribution and Joint Posterior Density*

Since we don't have prior information about α, β and γ, we use weakly informative prior distributions. Assume they are distributed independently, then $p(\theta) = p(\alpha, \beta, \gamma) = p(\alpha)p(\beta)p(\gamma)$, where

$$\begin{aligned} &p(\alpha) \propto 1 \\ &p(\beta) \propto |\Sigma_\beta|^{-1/2}exp(-\frac{1}{2}(\beta-\beta_0)^T\Sigma_\beta^{-1}(\beta-\beta_0)) \\ &p(\gamma) \propto |\Sigma_\gamma|^{-1/2}exp(-\frac{1}{2}(\gamma-\gamma_0)^T\Sigma_\gamma^{-1}(\gamma-\gamma_0)) \end{aligned}$$

with specified $\Sigma_\beta, \Sigma_\gamma, \beta_0, \gamma_0$.

The posterior distribution for θ will be:

$$\begin{aligned} p(\theta|x,w,y) = p(\alpha,\beta,\gamma|x,w,y) &\propto \Pi_{i=1}^{n}(\phi_i + (1-\phi_i)f(0;\mu_i,\alpha))^{I_{y_i=0}}((1-\phi_i)f(y_i;\mu_i,\alpha))^{I_{y_i>0}} \\ &\times |\Sigma_\beta|^{-1/2} exp(-\frac{1}{2}(\beta-\beta_0)^T\Sigma_\beta^{-1}(\beta-\beta_0)) \\ &\times |\Sigma_\gamma|^{-1/2} exp(-\frac{1}{2}(\gamma-\gamma_0)^T\Sigma_\gamma^{-1}(\gamma-\gamma_0)) \end{aligned}$$

Therefore, the full conditional posterior distributions for α, β and γ are

$$p(\alpha|x,w,y,\beta,\gamma) \propto \Pi_{i=1}^{n}(\phi_i + (1-\phi_i)f(0;\mu_i,\alpha))^{I_{y_i=0}}((1-\phi_i)f(y_i;\mu_i,\alpha))^{I_{y_i>0}}$$

$$\begin{aligned} p(\beta|x,w,y,\alpha,\gamma) &\propto \Pi_{i=1}^{n}(\phi_i + (1-\phi_i)f(0;\mu_i,\alpha))^{I_{y_i=0}}((1-\phi_i)f(y_i;\mu_i,\alpha))^{I_{y_i>0}} \\ &\times |\Sigma_\beta|^{-1/2} exp(-\frac{1}{2}(\beta-\beta_0)^T\Sigma_\beta^{-1}(\beta-\beta_0)) \end{aligned}$$

$$\begin{aligned} p(\gamma|x,w,y,\alpha,\beta) &\propto \Pi_{i=1}^{n}(\phi_i + (1-\phi_i)f(0;\mu_i,\alpha))^{I_{y_i=0}}((1-\phi_i)f(y_i;\mu_i,\alpha))^{I_{y_i>0}} \\ &\times |\Sigma_\gamma|^{-1/2} exp(-\frac{1}{2}(\gamma-\gamma_0)^T\Sigma_\gamma^{-1}(\gamma-\gamma_0)) \end{aligned}$$

By using Metropolis algorithm, we can generate samplers of these full conditional distributions to get the Bayesian estimation of $\theta = (\alpha, \beta, \gamma)$.

3.2 *Bayesian Model Comparison*

In this project, we will use two computationally tractable Bayesian procedures to compare competing models: the deviance information criterion (DIC) and the conditional predictive ordinate (CPO) statistics.

The DIC is defined as $DIC = \bar{D}(\theta) + p_D$, where $\bar{D}(\theta) = E(D(\theta)|y)$. $D(\theta)$ is the deviance and p_D is the effective number of parameters, defined as $p_D = E(D(\theta)) - D(E(\theta))$. Let $\theta^{(1)}, \cdots, \theta^{(K)}$ be a sample of size K from $p(\theta|x,w,y)$ after the burn-in. Then the DIC will be approximately

$$\widehat{DIC} = \bar{D} + \hat{p}_D = 2\bar{D} - \hat{D}$$

where $\hat{D} = D((1/K)\sum_{t=1}^{K}\theta^{(t)})$. For model fit comparisons, a model with smaller DIC should be preferred to a model with larger DIC.

The CPO statistic is defined as the predictive density of the i^{th} case, where the data doesn't contain the i^{th} case. Let $D = \{x, w, y\}$ and $D_{(i)} = \{x_{(i)}, w_{(i)}, y_{(i)}\}$ be the same quantity but without the i^{th} case. Then, the CPO statistics for the i^{th} case will be

$$CPO_i = p(y_i|D_{(i)}) = \int p(y_i|\theta)p(\theta|D_{(i)})d\theta = (\int \frac{p(\theta|D)}{p(y_i|\theta)}d\theta)^{-1}$$

where $p(\theta|D_{(i)})$ is the posterior density of θ given $D_{(i)}$. By using Metropolis algorithm, a Monte Carlo estimate of CPO_i is

$$\widehat{CPO_i} = [\frac{1}{K}\sum_{t=1}^{K}\frac{1}{p(y_i|\theta^{(t)}, X_i, W_i)}]^{-1}$$

Then the statistic $B = \sum_{i=1}^{n}\log(\widehat{CPO_i})$ can be used to compare the fit of two models. A model with larger B value should be preferred to a model with smaller B value.

3.3 *Bayesian Case Influence Diagnostics*

Let $L(\theta|D)$ be the likelihood on the full data and $L(\theta|D_{(i)})$ be the likelihood on the data without the i^{th} case. The posterior distributions of $\theta = (\alpha, \beta^T, \gamma^T)^T$ for the full data and data without the i^{th} case will be

$$p(\theta|D) \propto L(\theta|D)p(\theta)p(\theta|D_{(i)}) \propto L(\theta|D_{(i)})p(\theta)$$

For simplicity, let P denote $p(\theta|D)$, $P_{(i)}$ denote $p(\theta|D_{(i)})$ and $K(P, P_{(i)})$ denote the K-L divergence between P and P_i, then

$$K(P, P_{(i)}) = \int p(\theta|D) \log(\frac{p(\theta|D)}{p(\theta|D_{(i)})}) d\theta$$

Therefore, $K(P, P_{(i)})$ measures the effect of deleting the i^{th} case from the full data on the joint posterior distribution of θ and its expression for the ZIGP regression model is

$$\begin{aligned} K(P, P_{(i)}) &= \log E_\theta\{[p(y_i|\theta, X_i, W_i)]^{-1}|D\} + E_\theta\{\log[p(y_i|\theta, X_i, W_i)]|D\} \\ &= -\log(CPO_i) + E_\theta\{\log[p(y_i|\theta, X_i, W_i)]|D\} \end{aligned}$$

By using Metropolis algorithm, a Monte Carlo estimate of $K(P, P_{(i)})$ can be expressed as

$$K(P, P_{(i)}) = \log[\frac{1}{K}\sum_{t=1}^{K} \frac{1}{p(y_i|\theta^{(t)}, X_i, W_i)}] + \frac{1}{K}\sum_{t=1}^{K} \log[p(y_i|\theta^{(t)}, X_i, W_i)],\ i = 1, \cdots, n$$

Data analysis

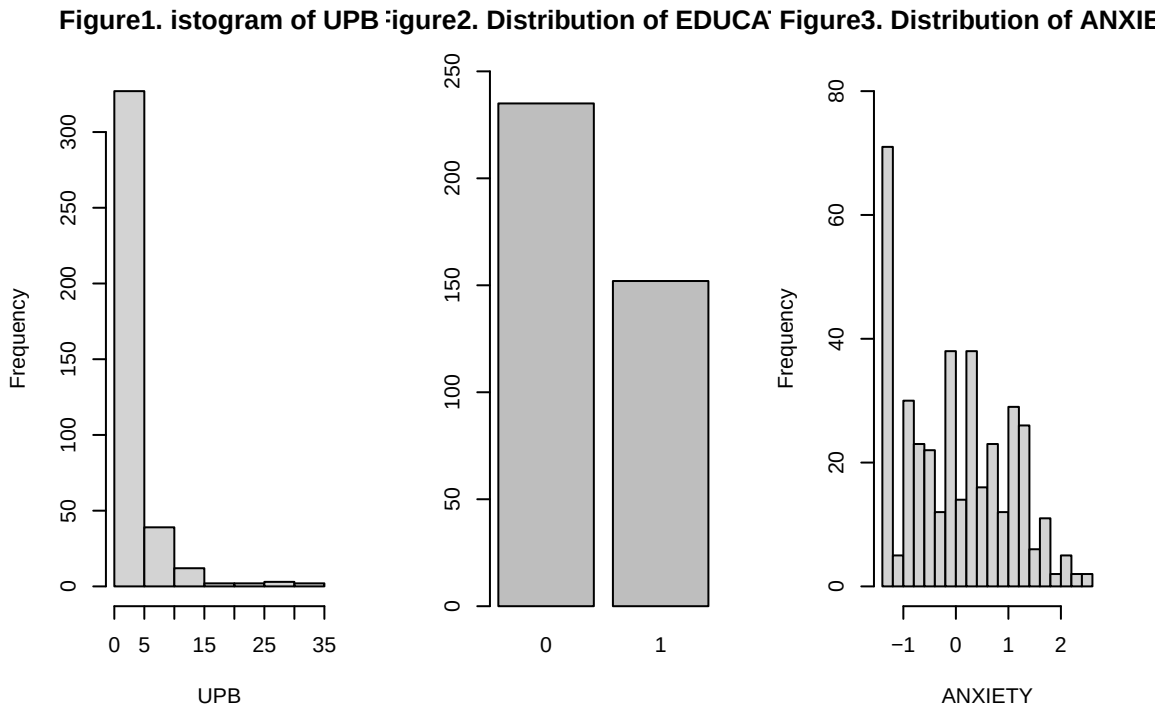
In this section, we will conduct ZIGP and ZIP regression model on the data set.

Data Introduction

We now illustrate the methodology using the "couple" dataset about real psychology example. it contains three random variables education level' (X1), 'level of anxious attachment' (X2) and number of unwanted pursuit behavior (UPB) perpetrations (y) . One of the objectives of this study is to explore the impact of 'education level' and 'level of anxious attachment' on the number of unwanted pursuit behavior (UPB) perpetrations in the context of couple separation trajectories. For more details on this data set, please referred to [reference2].

Data visualization

Here we plot the distribution of all variables in the data set.



Model fitting

Firstly, we assume that:

$$log(\mu_i) = \beta_{10} + \beta_{11}X_{1i} + \beta_{2i}X_{2i}$$
$$logit(\phi_i) = \gamma_{10} + \gamma_{11}X_{1i} + \gamma_{12}X_{2i}$$

for both models. We denote $\beta = (\beta_{10}, \beta_{11}, \beta_{12})^T$, $\gamma = (\gamma_{10}, \gamma_{11}, \gamma_{12})^T$ and $\theta = (\alpha, \beta^T, \gamma^T)^T$.

As for the prior distribution:

$$p(\alpha) \propto 1$$
$$p(\beta) \propto dnorm(\beta_0, \Sigma_\beta)$$
$$p(\gamma) \propto dnorm(\gamma_0, \Sigma_\gamma)$$

We took $\beta_0 = 0$, $\Sigma_\beta = 1000I_3$, $\gamma_0 = 0$ and $\Sigma_\gamma = 1000I_3$. These prior distribution are weakly informative and close to non-informative ones.

We used Metropolis method to conduct the MCMC process. We chose to update the parameters one by one like the gibbs sampler.The proposal function for ith parameter in *theta* would be $J(\theta|\theta_i^{(s)}) = N(\theta_i^{(s)}, 1)$. So the fitting steps were:

For ith parameter in total k parameters:

1. Repeatedly Choose $\theta_i^* \sim J(\theta|\theta_i^{(s)})$, $\theta^* = (\theta_1^{(s+1)}, \cdots \theta_{i-1}^{(s+1)}, \theta_i^*, \theta_{i+1}^{(s)}, \cdots, \theta_k^{(s)})$, until all $1 + \alpha y \geq 0$ and all $1 + \alpha\mu \geq 0$.
2. $log(r) = log(p(\theta^*|y, x)) + log(p(\theta^*)) - log(p(\theta^{(s)}|y, x)) - log(p(\theta^{(s)}))$.
3. If $log(u) < log(r)$, $\theta = \theta^*$.

We ran the MCMC chain for 50,000 iterations for each parameter, discarding first 10,000 as a burn-in to ensure the convergence. Also, to reduce the correlation, we considered a spacing size of 20, obtaining the total sample of 2000 size.

Estimation of parameters

Table 1 and table 2 presented the parameter estimation of both method, where Mean is the mean of posterior distribution of parameters, SD is the standard deviance and HPD is the 95% highest density interval:

Table 1: TABLE1. Bayesian estimates of parameters for ZIGP model

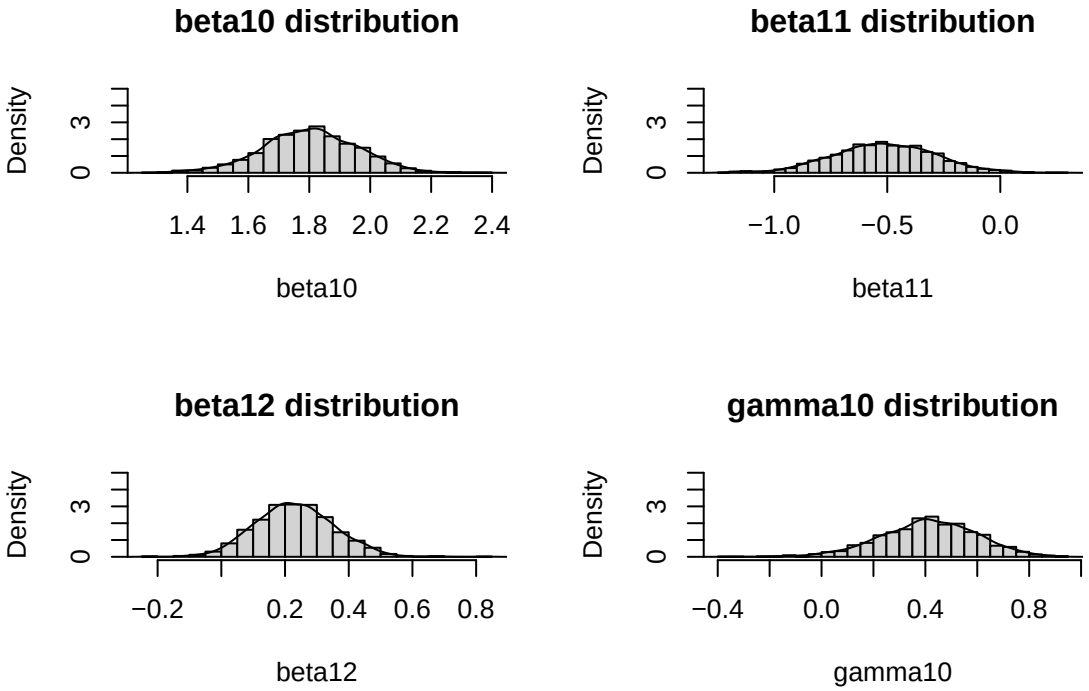
parameter	Mean	SD	HPD2.5.	HPD97.5.
beta10	1.8014	0.1550	1.4834	2.0918
beta11	-0.5147	0.2249	-0.9425	-0.0628
beta12	0.2308	0.1230	-0.0053	0.4620
gamma10	0.4122	0.1844	0.0522	0.7742
gamma11	-0.4139	0.2816	-0.9545	0.1350
gamma12	-0.5239	0.1383	-0.7839	-0.2561
alpha	0.3569	0.0719	0.2390	0.5112

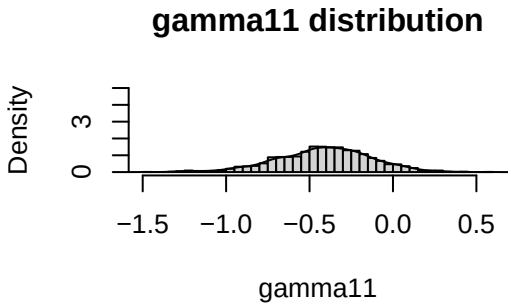
Table 2: TABLE2. Bayesian estimates of parameters for ZIGP model

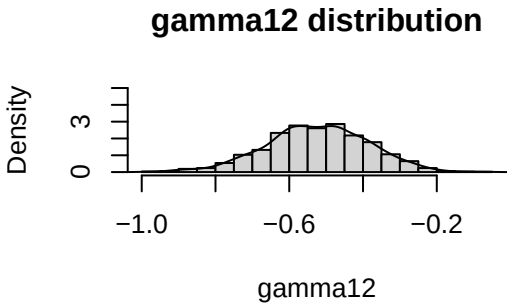
parameter	Mean	SD	HPD2.5.	HPD97.5.
beta10	1.9220	0.0430	1.8353	2.0011
beta11	-0.3570	0.0703	-0.4928	-0.2183
beta12	0.1334	0.0352	0.0621	0.1973
gamma10	0.6705	0.1470	0.3997	0.9667

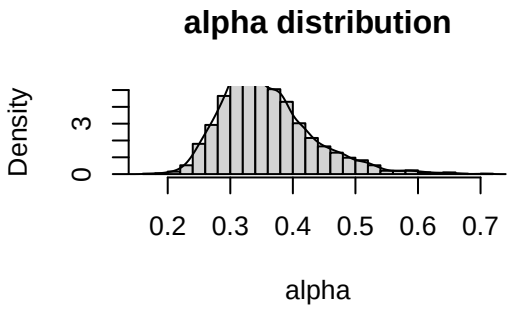
parameter	Mean	SD	HPD2.5.	HPD97.5.
gamma11	-0.2266	0.2242	-0.6678	0.2030
gamma12	-0.4841	0.1142	-0.6992	-0.2638

Here are the posterior distribution of parameters in ZIGP model.









Goodness of fit

We computed DIC and B statistics to compare the performance of two models. The results are in table 3:

Table 3: Table3. Bayesian comparsion for ZIGP and ZIP model

Model	DIC	B
ZIP	1617.128	-816.5520
ZIGP	1268.055	-634.4684

We can see that ZIGP model has smaller DIC and larger B statistics, which means the ZIGP outperforms the ZIP model.

Influential observations and K-L divergence

Figure 4 and figure 5 show the meansures of K-L divergence presented in previous section, based on the sample of the posterior distribution s of parameters of ZIP and ZIGP models respectively. We can see that case 120, 238, 292, 300, 323 and 335 are most influential cases for ZIP model. So we compare K-L divergence for both models for these cases. The results are in table 4:

Figure4.

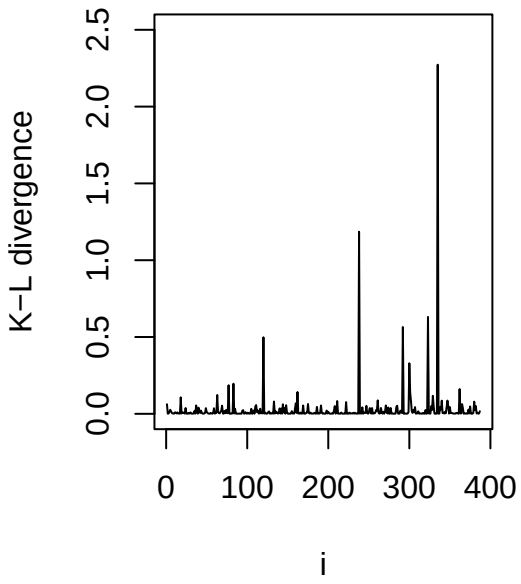


Figure5.

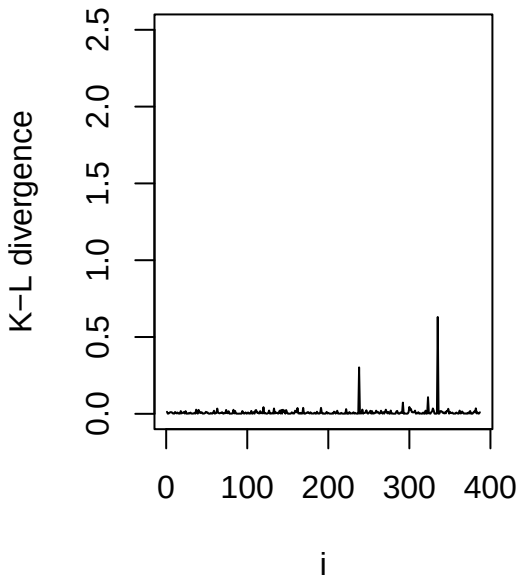


Table 4: Table4. Most influential case

Case	ZIP.K	ZIGP.K
120	0.4982	0.0433
238	1.1852	0.3022
292	0.5652	0.0731
300	0.3293	0.0444
323	0.6306	0.1084
335	2.2729	0.6304

We can see that the cases 238 and 335 are identified as potential influence points in both models, and cases 120, 292, 300 and 323 are only detected as influential in ZIP model. In general, these influential points correspond to large number of UPB values. Further, from table 4, figure 4 and figure 5 show that all cases for the ZIGP model tend to have smaller divergence than ZIP model.

Conclusion and Discussion

Conclusion

In this work, we programmed to implement MCMC methods as an alternative way to get Bayesian inference for the ZIGP regression model and conducted data analysis. Additionally, in order to assess the sensitivity of the Bayesian estimates, we consider Bayesian case influence diagnostics based on the K–L divergence. One real example from the psychological field is used to compare the sensitivity of the parameters from ZIGP and ZIP regression models. In our study,we use R program to analysis the same data set and approach the same results in the end. Eventually, we get the similar results like the original paper. We found that ZIGP method outperformed the ZIP method in terms of DIC statistics, B statistics and K-L divergence.

Limitation

In the data analysis, we used weakly informative priors and non-informative priors for all parameters like the original paper. We think these kinds of prior might be not helpful in practical data analysis. The ZIGP model would convergence faster if we have better priors.

Further Work

We could look over more papers and materials to find better priors for this kind of data. Also we could try more priors to find out better ones.

References

Feng-Chang Xie, Jin-Guan Lin & Bo-Cheng Wei (2014) Bayesian zero-inflated generalized Poisson regression model: estimation and case influence diagnostics, Journal of Applied Statistics, 41:6,1383-1392, DOI: 10.1080/02664763.2013.871508

T. Loeys, B. Moerkerke, and O.D. Smet, The analysis of zero-inflated count data: Beyond zero-inflated Poisson regression, British J. Math. Statist. Psychol. 65 (2012), pp. 163–180. doi: 10.1111/j.2044-8317.2011.02031.x

Appendix

```
library(mvtnorm)
library(HDInterval)
knitr::opts_chunk$set(fig.width=6, fig.height=4)
# read data and initialization
df = read.table("~/Desktop/Courses/619/project/data.txt", header = TRUE)
idx0 = df[, 1] == 0
X = cbind(1, as.matrix(df[, 2:3]))
Y = c(df[, 1])
sd = 1
beta0 = c(0, 0, 0)
sigmaB = 1000* diag(c(1, 1, 1))
gamma0 = c(0, 0, 0)
sigmaG = 1000*diag(c(1, 1, 1))

# density fucntion of GP
GP_f = function(y, x, beta, alpha = 0) {
  mu = exp(x %*% beta)
  return((mu/(1 + alpha*mu))^y * (1 + alpha*y)^(y-1) * exp(-mu*(1 + alpha*y) / (1 + alpha*mu)) / factor
}

# density function of Possion
P_f = function(y, x, beta) {
  mu = x %*% beta
  return(dpois(y, mu))
}

# log prior density of beta
logp_beta = function(x) {
  return(dmvnorm(x, beta0, sigmaB, log = TRUE))
}

# log prior density of gamma
logp_gamma = function(x) {
  return(dmvnorm(x, gamma0, sigmaG, log = TRUE))
}

# log likelihood

log_lik = function(alpha, beta, gamma) {
  rho = exp(X %*% gamma) / (1 + exp(X %*% gamma))
  ret = sum(log(rho[idx0] + (1 - rho[idx0]) * GP_f(Y[idx0], X[idx0, ], beta, alpha))) + sum(log(1 - rho
  return(ret)
}

# probability of one sample

prob = function(y, x, alpha, beta, gamma) {
  rho = exp(x %*% gamma) / (1 + exp(x %*% gamma))
  if (y == 0) {
    return(rho + (1 - rho) * GP_f(y, x, beta, alpha))
  }
```

```
  else {
    return((1 - rho) * GP_f(y, x, beta, alpha))
  }
}


prob_all = function(alpha, beta, gamma) {
  rho = exp(x %*% gamma) / (1 + exp(x %*% gamma))
  ret = rep(0, length(Y))
  ret[idx0] = rho[idx0] + (1 - rho[idx0])*f(Y[idx0], X[idx0, ], beta, alpha)
  ret[!idx0] = (1 - rho[!idx0])*f(Y[!idx0], X[!idx0, ], beta, alpha)
  return(ret)
}


#full conditional in GP regression
logfull_alpha = function(alpha, beta, gamma) {
  rho = exp(X %*% gamma) / (1 + exp(X %*% gamma))
  ret = sum(log(rho[idx0] + (1 - rho[idx0]) * GP_f(Y[idx0], X[idx0, ], beta, alpha))) + sum(log(GP_f(Y[
  return(ret)
}

logfull_beta = function(alpha, beta, gamma) {
  ret = logfull_alpha(alpha, beta, gamma) - 0.5*t(beta - beta0) %*% solve(sigmaB) %*% (beta - beta0)
  return(ret)
}

logfull_gamma = function(alpha, beta, gamma) {
  rho = exp(X %*% gamma) / (1 + exp(X %*% gamma))
  ret = sum(log(rho[idx0] + (1 - rho[idx0]) * GP_f(Y[idx0], X[idx0, ], beta, alpha))) + sum(log(1 - rho
  ret = ret - 0.5*t(gamma - gamma0) %*% solve(sigmaG) %*% (gamma - gamma0)
  return(ret)
}

# fit GP regression

check = function(alpha, beta, gamma) {
  mu = exp(X %*% beta)
  if (any(1 + alpha*mu < 0) | any((1 + alpha*Y) < 0)) {
    return(FALSE)
  }
  return(TRUE)
}

update_alpha = function(alpha, beta, gamma) {
  alpha_star = rnorm(1, alpha, sd)
  while (TRUE) {
    if (check(alpha_star, beta, gamma)){
      break
    }
    alpha_star = rnorm(1, alpha, sd)
  }
  log_r = logfull_alpha(alpha_star, beta, gamma) - logfull_alpha(alpha, beta, gamma)
```

```
  if (log(runif(1)) < log_r) {
    alpha = alpha_star
  }
  return(alpha)
}

update_beta = function(alpha, beta, gamma) {
  for (i in 1:3) {
    new_beta = beta
    new_beta[i] = rnorm(1, beta[i], sd)
    while (TRUE) {
      if (check(alpha, new_beta, gamma)){
        break
      }
      new_beta[i] = rnorm(1, beta[i], sd)
    }

    log_r = logfull_beta(alpha, new_beta, gamma) + dnorm(new_beta[i], 0, sqrt(1000), log = TRUE) - logfu
    if (log(runif(1)) < log_r) {
      beta[i] = new_beta[i]
    }
  }
  return(beta)
}

update_gamma = function(alpha, beta, gamma) {
  for (i in 1:3) {
    gammai_star = rnorm(1, gamma[i], sd)
    new_gamma = gamma
    new_gamma[i] = gammai_star
    while (TRUE) {
      if (check(alpha, beta, new_gamma)){
        break
      }
      new_gamma[i] = rnorm(1, gamma[i], sd)
    }

    log_r = logfull_gamma(alpha, beta, new_gamma) + dnorm(new_gamma[i], 0, sqrt(1000), log = TRUE) - log
    if (log(runif(1)) < log_r) {
      gamma[i] = new_gamma[i]
    }
  }
  return(gamma)
}


fit_GP = function(N){
  ret = NULL
  alpha = runif(1)
  beta = c(rmvnorm(1, beta0, sigmaB/1000))
  gamma = c(rmvnorm(1, gamma0, sigmaG/1000))

  while (TRUE) {
```

```
    if (check(alpha, beta, gamma)) {
      break
    }
    alpha = runif(1)
    beta = c(rmvnorm(1, beta0, sigmaB/1000))
    gamma = c(rmvnorm(1, gamma0, sigmaG/1000))
  }
  for (i in 1:N) {
    alpha = update_alpha(alpha, beta, gamma)
    beta = update_beta(alpha, beta, gamma)
    gamma = update_gamma(alpha, beta, gamma)
    if (i %% 20 == 0) {
      ret = rbind(ret, c(alpha, beta, gamma))
    }
  }
  return(ret)
}
# fit Possion regression
fit_P = function(N) {
  ret = NULL
  beta = c(rmvnorm(1, beta0, sigmaB/1000))
  gamma = c(rmvnorm(1, gamma0, sigmaG/1000))

  while (TRUE) {
    if (check(0, beta, gamma)) {
      break
    }
    beta = c(rmvnorm(1, beta0, sigmaB/1000))
    gamma = c(rmvnorm(1, gamma0, sigmaG/1000))
  }
  for (i in 1:N) {
    beta = update_beta(0, beta, gamma)
    gamma = update_gamma(0, beta, gamma)
    if (i %% 20 == 0) {
      ret = rbind(ret, c(beta, gamma))
    }
  }
  return(ret)

}
#ret = fit_P(1000)

# DIC of GP regression
DIC_GP = function(samples) {
  mean_sample = colMeans(samples)
  D_hat = -2*log_lik(mean_sample[1], mean_sample[2:4], mean_sample[5:7])
  res = rep(0, nrow(samples))
  for (i in 1: nrow(samples)) {
    res[i] = -2*log_lik(samples[i, 1], samples[i, 2:4], samples[i, 5:7])
  }
  D_bar = mean(res)
  return(2*D_bar - D_hat)
}
```

```r
#DIC_GP(samples)

# DIC of Possion regression
DIC_P = function(samples) {
  mean_sample = colMeans(samples)
  D_hat = -2*log_lik(0, mean_sample[1:3], mean_sample[4:6])
  res = rep(0, nrow(samples))
  for (i in 1: nrow(samples)) {
    res[i] = -2*log_lik(0, samples[i, 1:3], samples[i, 4:6])
  }
  D_bar = mean(res)
  return(2*D_bar - D_hat)
}

# B stats of GP regression

B_GP = function(samples) {
  cpo = rep(0, nrow(df))
  n = nrow(samples)
  for (i in 1: nrow(df)) {
    curr = rep(0, n)
    for (j in 1:n) {
      curr[j] = 1/ prob(Y[i], X[i, ], samples[j, 1], samples[j, 2:4], samples[j, 5:7])
    }
    cpo[i] = 1/mean(curr)
  }
  return(sum(log(cpo)))
}

# B stats of Possion regression
B_P = function(samples) {  cpo = rep(0, nrow(df))
  n = nrow(samples)
  for (i in 1: nrow(df)) {
    curr = rep(0, n)
    for (j in 1:n) {
      curr[j] = 1/ prob(Y[i], X[i, ], 0, samples[j, 1:3], samples[j, 4:6])
    }
    cpo[i] = (mean(curr)) ^ (-1)
  }
  return(sum(log(cpo)))
}

# K stats of GP regression
K_GP = function(samples, i) {
  n = nrow(samples)
  curr = rep(0, n)
  p = rep(0, n)
  for (j in 1:n) {
    curr[j] = 1/ prob(Y[i], X[i, ], samples[j, 1], samples[j, 2:4], samples[j, 5:7])
    p[j] = log(prob(Y[i], X[i, ], samples[j, 1], samples[j, 2:4], samples[j, 5:7]))
  }
  ret = log(mean(curr)) + mean(p)
  return(ret)
```

```
}

# K stats of Possion regression
K_P = function(samples, i) {
  n = nrow(samples)
  curr = rep(0, n)
  p = rep(0, n)
  for (j in 1:n) {
    curr[j] = 1/ prob(Y[i], X[i, ], 0, samples[j, 1:3], samples[j, 4:6])
    p[j] = log(prob(Y[i], X[i, ], 0, samples[j, 1:3], samples[j, 4:6]))
  }
  ret = log(mean(curr)) + mean(p)
  return(ret)
}

# run MCMC
set.seed(2022)
ret_GP = fit_GP(50000)
ret_P = fit_P(50000)
ret_GP = ret_GP[501:2500, ]
ret_P = ret_P[501:2500, ]

# get statistics
ret_GP1 = cbind.data.frame(ret_GP[, 2:7], ret_GP[, 1])
colnames(ret_GP1) = 1:7

means = round(colMeans(ret_GP1), 4)
sds = round(apply(ret_GP1, 2, sd), 4)
CI = round(hdi(ret_GP1), 4)
table1 = data.frame("parameter" = c("beta10", "beta11", "beta12", "gamma10", "gamma11", "gamma12", "alp

means = round(colMeans(ret_P), 4)
sds = round(apply(ret_P, 2, sd), 4)
CI = round(hdi(ret_P), 4)
table2 = data.frame("parameter" = c("beta10", "beta11", "beta12", "gamma10", "gamma11", "gamma12"), "Mea

dic_gp = round(DIC_GP(ret_GP), 4)
dic_p = round(DIC_P(ret_P), 4)
b_gp = round(B_GP(ret_GP), 4)
b_p = round(B_P(ret_P), 4)
table3 = data.frame("Model" = c("ZIP", "ZIGP"), "DIC" = c(dic_p, dic_gp), "B" = c(b_p, b_gp))

ks_gp = rep(0, 387)
ks_p = rep(0, 387)
for (i in 1:387) {
  ks_gp[i] = K_GP(ret_GP, i)
  ks_p[i] = K_P(ret_P, i)
}

idx = c(120, 238, 292, 300, 323, 335)
table4 = data.frame("Case" = idx, "ZIP K" = round(ks_p[idx], 4), "ZIGP K" = round(ks_gp[idx], 4))

par(mfrow = c(1, 3))
```

```r
hist(Y, main = "Figure1. istogram of UPB", xlab = "UPB")
barplot(table(X[, 2]), ylim = c(0, 250), main = "Figure2. Distribution of EDUCATION")
hist(X[, 3], ylim = c(0, 80), breaks = 20, main = "Figure3. Distribution of ANXIETY", xlab = "ANXIETY")

knitr::kable(table1, caption = "TABLE1. Bayesian estimates of parameters for ZIGP model")
knitr::kable(table2, caption = "TABLE2. Bayesian estimates of parameters for ZIGP model")

names = c("beta10", "beta11", "beta12", "gamma10", "gamma11", "gamma12", "alpha")
for (i in 1:7) {
  if (i %% 4 == 1) {
    par(mfrow = c(2, 2))
  }
  hist(ret_GP1[, i], freq = F, ylim = c(0, 5), xlab = names[i], main = paste0(names[i], " distribution"))
  lines(density(ret_GP1[, i]))
}
knitr::kable(table3, caption = "Table3. Bayesian comparsion for ZIGP and ZIP model")
par(mfrow = c(1, 2))
plot(1:387, ks_p, type = "l", xlab = "i", ylab = "K-L divergence", ylim = c(0, 2.5), main = "Figure4. ")
plot(1:387, ks_gp, type = "l", xlab = "i", ylab = "K-L divergence", ylim = c(0, 2.5), main = "Figure5. ")
knitr::kable(table4, caption = "Table4. Most influential case")
```